SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

1. Introduction:

This project introduces an automated solution to the manual, inefficient, and error-prone procurement processes found in traditional supply chain management. By deploying a cluster of intelligent agents based on the GLM-4 architecture, the system automates the entire pipeline from requirement understanding to supplier matching and professional email drafting.

1.1. Purpose:

The System Analysis Documentation highlights the technical scope and design-related decisions behind the development of the Procurement Agent System.

The key elements covered are as follows:

- Architecture: A lightweight Flask-based web application utilizing a multi-agent orchestration workflow.
- Data Flows: Demonstrates how raw procurement requests (text/images) are transformed into structured JSON, matched against supplier databases, and outputted as business communications.
- Model Process: Details the end-to-end workflow of core features: requirement extraction, strategic sourcing with risk assessment, and automated drafting.
- Role of Reference: This document serves as a technical reference for developers to ensure architectural consistency and for testers to validate the logic of agent transitions.

1.2. Background:

In modern global trade, procurement officers struggle with multi-language requests, inconsistent supplier data, and hidden geographical risks.

Previous Version: Existing methods rely on manual Excel comparisons and human-led risk assessment, which are slow and lack real-time data integration.

Solution: Project Moonlight replaces manual intervention with three specialized agents: Requirement Extractor, Sourcing Analyst, and Procurement Coordinator.

1.3. Target Stakeholder

Stakeholders	Roles	Expectations
Procurement Officers	Primary User	Professionals responsible for sourcing products and communicating with vendors. They use the system to automate data entry and email drafting.

© UMHackathon 2026

Supply Chain Managers	Process Supervisor	Oversee the procurement pipeline. They use the system's "Geo-Risk" and "Fraud Detection" insights to ensure compliance and cost-efficiency.
IT/Systems Administrators	Technical Maintainer	Responsible for maintaining the Flask server, updating the suppliers.csv database, and managing Zhipu AI API configurations.
External Vendors/Suppliers	External Recipient	Receivers of the automated inquiries. They benefit from clearer, professional, and structured communication generated by the agent.

© UMHackathon 2026

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. Overview:

Type	Details
System	A sequential AI pipeline where tasks are decomposed into specialized roles. It utilizes a "Chain-of-Thought" process where the output of one agent (e.g., Requirements) serves as the immutable context for the next (e.g., Sourcing).
Architecture	The system operates on an asynchronous-style event loop managed by a Flask backend. It triggers specific logic branches based on user "Intents" (NEW_ORDER vs. UPDATE) and handles data flow between the UI and the Zhipu AI GLM-4 inference engine.

System Type:
This project is primarily structured as a Multi-Agent Orchestration System in which there is 1 unified interface, Procurement Web-based Dashboard. This client communicates via a Flask Backend API layer that orchestrates three independent agentic modules: Requirement Extraction Service, Strategic Sourcing Service, and Drafting & Communication Service. All services are integrated with the Zhipu AI GLM-4 Inference Engine, and the system utilizes a local CSV-based repository (suppliers.csv) for high-speed supplier matching and risk validation.

Architecture Type:
The system follows an Event-Driven Micro-Service Architecture designed for a sequential pipeline. When a user triggers an event (uploading a document or entering text), the Backend API manages the state and hand-off between services. The Requirement Extraction Service first structures raw data, which is then passed to the Sourcing Service for fuzzy database matching and geographic risk calculation. Finally, the Communication Service synthesizes this data into multi-lingual business drafts. This modular architecture allows each agent to operate independently while sharing a common context through JSON-based data exchange.

2.1.2 LLM as Service Layer

The Procurement Agent System is primarily structured as a Multi-Agent Orchestration System in which there is 1 unified interface: The Procurement Web-based Dashboard. This client communicates via a Flask Backend API layer that orchestrates three independent agentic modules: Requirement Extraction Service, Strategic Sourcing Service, and Drafting & Communication Service. All services are integrated with the Zhipu AI GLM-4 Inference

© UMHackathon 2026

Engine as a core service layer, while the system utilizes a local CSV-based repository (suppliers.csv) for ground-truth matching and risk validation.

2.1.3 Dependency Diagram

a. Prompt Construction: Prompts are dynamically generated in the requirement_extractor.py and procurement_coordinator.py modules. They use structured f-string templates that define a "Procurement Specialist" persona, injected with raw user input and matched supplier data.

b. Context Window: The context window contains the System Role (persona), the user's raw input (text/extracted OCR), and for the final stage, the JSON-formatted matching results from the sourcing layer to ensure the LLM drafts emails based on actual database facts.How is your system receiving, parsing the responses and passing to the next system component?

c. Response Parsing: The system utilizes GLM-4's json_object response format. The Backend receives the string response, validates it via json.loads(), and passes the resulting Python dictionary as an object to the next function in the pipeline.Also, do outline all the major API calls between the system components [e.g. Customer Mobile App calls "Amazon API Gateway" for Order Service]

d. Token & Input Management: For visual inputs (images), GLM-4V is used for visual tokenization. Large text inputs are handled by specialized parsers (python-docx/PyPDF2) before reaching the API to ensure they fit within the 128k context window.

e. Major API Calls: Dashboard calls POST /run-pipeline to the Flask Backend.
Flask Backend calls the GLM-4V API for visual analysis of documents.
Requirement Service calls GLM-4 API for semantic parsing.
Sourcing Service communicates with the Local File System to query suppliers.csv.

f. Component Interaction: The Order Service logic communicates with the Local CSV Storage to ensure that product names and prices are verified against real inventory data before the Communication Service generates a final draft.

g. Deployment: The system is designed as a distributed architecture where the Web/API Backend runs on a local or cloud server (e.g., AWS EC2), while the Inference Layer is served externally via Zhipu AI's distributed GPU cluster.

2.2.2. Sequence Diagram

This walk-through demonstrates the system operation from a raw user request to the final email generation:

Step 1: Input Ingestion

© UMHackathon 2026

- The user uploads a photo of a handwritten requisition or types a request (e.g., "Need 50 units of Aura Buds").
- Frontend sends the data to the Flask Backend.

Step 2: Requirement Extraction (Agent 1)

- The Backend calls the Requirement Extraction Service.
- If an image is provided, GLM-4V performs OCR.
- GLM-4 parses the text into a structured JSON (Intent: NEW_ORDER, Product: Aura Buds, Qty: 50).

Step 3: Strategic Sourcing (Agent 2)

- The Backend passes the JSON to the Sourcing Service.
- The service queries suppliers.csv. It uses Jaccard Similarity to match "Aura Buds" with "Aura Pro Wireless Earbuds."
- It calculates Geographic Risk (e.g., Supplier in China vs. KL) and Fraud Risk (Price comparison).

Step 4: Draft Synthesis (Agent 3)

- The Backend passes the matched supplier data to the Drafting Service.
- GLM-4 generates a professional business email in the user's detected language.

Step 5: Review & Delivery

- The Backend returns the complete "Thinking Process" and the email draft to the Dashboard.
- The user reviews the draft and triggers the send_emails route to finalize the communication.

2.2. Technological Stack

2.2.1. Frontend

Vanilla JavaScript, HTML5, and CSS3 – A responsive web-based dashboard designed for desktop procurement environments. This stack was chosen for its zero-overhead integration with the Flask backend, allowing for real-time "Thinking Process" visualizations and dynamic rendering of multi-agent outputs without the complexity of heavy frontend frameworks.

2.2.2. Backend

Flask (Python) – A lightweight WSGI web application framework used to orchestrate the multi-agent pipeline. Flask serves as the central hub that receives procurement requests, manages the sequential hand-off between the three specialized agents (Extraction, Sourcing, and Coordination), and interfaces with the Zhipu AI SDK.

2.2.3. Database

CSV (Flat-file Database) – The system utilizes suppliers.csv as a high-speed, local relational-

© UMHackathon 2026

style database. This allows for low-latency fuzzy matching and geographic risk calculations. The data layer includes a custom sanitization engine to handle multi-language encoding (UTF-8-SIG) and dynamic column mapping for supplier details, pricing, and origin countries.

- Cloud/Deployment
  Inference Layer: Zhipu AI Cloud Platform – Hosts the GLM-4 (Text) and GLM-4V (Vision) models, providing the distributed GPU power necessary for natural language understanding and OCR.

- Application Server: Local/Cloud Host (e.g., PythonAnywhere or AWS EC2) – Used to host the Flask application and the static supplier database.

- SMTP Service: Integrated via Python's smtplib to handle the final delivery of generated procurement inquiries to identified vendors.

2.3. Key Data Flows

2.3.1. Data Flow Diagram (DFD) Analysis

The data movement within the system follows a sequential pipeline designed to transform unstructured human intent into structured commercial actions.

2.3.1.1. Data Movement Overview

External Entity (User)→Process 1.0 (Ingestion): User provides input (Text/Image/PDF). If an image is detected, data flows to the External Service (GLM-4V) for OCR.
Process 1.0 →Process 2.0 (Extraction): Extracted text flows into the Requirement Service (GLM-4). The output is a structured Requirement JSON.
Process 2.0→Process 3.0 (Sourcing): Requirement JSON flows to the Sourcing Engine. This process fetches records from the Database (suppliers.csv).
Database→Process 3.0: Supplier records are retrieved and compared using Jaccard Similarity.
Process 3.0→Process 4.0 (Drafting): Validated matches and risk alerts flow into the Communication Service (GLM-4).
Process 4.0→External Entity (User): Final email drafts and the "Thinking Process" log are returned to the UI.Normalized Database Schema

2.3.1.2. Data Law Compliance (Avoidance of Black Holes):

To ensure no data "Black Holes" occur (where data enters a process but never leaves or is lost), the system implements State Persistence:
Input Validation: Every process checks for a non-null input before execution.
Fallback Logic: In requirement_extractor.py, if the GLM API fails, a "Default JSON" is generated so the pipeline does not terminate.
Logging: Every stage of the data flow is captured in the agent_results object, ensuring that even intermediate matching failures are reported back to the user.

2.3.2. Normalized Database Schema

© UMHackathon 2026

While the current implementation uses a flat-file CSV for rapid prototyping, the logical data model is designed according to 3rd Normal Form (3NF) principles to ensure scalability for a full SQL migration.

a. Logical Data Model (ERD) & 3NF Resolution:
To resolve the Many-to-Many (M:N) relationship between Procurement Requests and Suppliers (one request can involve multiple suppliers, and one supplier can fulfill multiple requests), we utilize a junction table.

1. Table: Products (1NF/2NF)
    - product_id (PK)
    - product_name
    - category_id (FK)
    - Ensures no repeating groups of product attributes.
2. Table: Suppliers (3NF)
    - supplier_id (PK)
    - supplier_name
    - contact_email
    - origin_country
    - Non-key attributes depend only on the Primary Key.
3. Table: Procurement_Orders (3NF)
    - order_id (PK)
    - user_id (FK)
    - order_date
    - intent_type (NEW_ORDER / UPDATE)
4. Table: Order_Items (Junction Table - 3NF)
    - order_item_id (PK)
    - order_id (FK)
    - product_id (FK)

© UMHackathon 2026

- supplier_id (FK)
- quantity
- quoted_price
- geo_risk_score
- Logic: This table resolves the M:N relationship, ensuring each specific item in a request is mapped to a specific verified supplier from the database.

3. Functional Requirements & Scope

3.1 Minimum Viable Product:

3.1.1 The following features represent the core technical milestones developed for the Procurement Agent System to be demonstrated by the end of UMHackathon 2026.

#	Features	Description
1	Multi-Modal Requirement Extraction	Users submit procurement needs via text, PDF, or images. The system utilizes GLM-4V for visual analysis and semantic parsing to convert unstructured requests into validated JSON objects (Intent, Product, Qty).
2	Intelligent Strategic Sourcing	The system automatically queries the suppliers.csv database using Jaccard Similarity fuzzy matching to identify the best-fit vendors even when product names do not perfectly match the user query.
3	Automated Risk & Fraud Assessment	For every potential match, the system calculates Geographic Risk (distance-based risk relative to KL) and Fraud Risk (identifying prices suspiciously below the market average) to protect the buyer.
4	Multi-Lingual Draft Synthesis	Based on extraction and sourcing results, the system generates professional business inquiry emails in the detected language (English, Malay, or Chinese) ready for review.
5	Agentic State Visualization	A specialized web dashboard that displays the "Thinking Process" of each agent, providing transparency into how the system moved from raw input to a final matching decision.

© UMHackathon 2026

3.2Non-Functional Requirements (NFRs)

These system qualities ensure the reliability, performance, and professional standard of the procurement pipeline.

Quality	Requirements	Implementation
Scalability	The system must handle multi-item procurement lists and large supplier databases without performance degradation.	Decoupled agent logic into modular services (agents.py); utilized efficient Python CSV sanitization and iterative list processing.
Reliability	The pipeline must not crash if the LLM returns malformed data or if a specific product is missing from the database.	Implemented Graceful Degradation logic in the extraction and sourcing modules with fallback "Default JSON" and RFI (Request for Information) templates.
Maintainability	The supplier database and risk thresholds must be easy to update by non-technical staff without modifying the source code.	Externalized data into suppliers.csv with a Dynamic Column Mapper that handles various header formats and encodings (UTF-8-SIG) automatically.
Token Latency	The API calls to the GLM inference engine should provide real-time feedback to prevent the user interface from appearing unresponsive.	Displaying a user-facing loading state and a step-by-step progress visualizer in the frontend while awaiting asynchronous agent responses.
Cost Efficiency	The average cost of tokens per user session must be minimized to maintain budget thresholds for high-volume procurement.	Prompt templates are optimized for fixed length; inputs are pre-parsed to remove noise, and JSON mode is enforced to prevent verbose/unstructured LLM output.

3.3 Out of Scope / Future Enhancements

While the current MVP demonstrates the core AI-driven pipeline from extraction to drafting, the following high-level features are planned for future development to complete the ecosystem:

a. Supplier Sentiment Monitor: An advanced NLP module designed to analyze historical communication and public data to monitor supplier sentiment. It will identify potential

© UMHackathon 2026

relationship strains or service quality drops by analyzing the tone of vendor responses and market reputation.

b. Agentic Negotiation: An autonomous negotiation layer where the agents can engage in multi-turn, back-and-forth communication with vendors. The agents will use predefined "Negotiation Playbooks" to counter-offer on prices, lead times, and payment terms to achieve the best TCO (Total Cost of Ownership).

c. Predictive Stockout and Replenishment: Integration with real-time inventory sensors and sales data to predict stockouts before they occur. The system will leverage time-series forecasting to trigger automatic "Replenishment Events," allowing the multi-agent pipeline to begin sourcing without human intervention.Monitor, Evaluation, Assumptions & Dependencies

4. Monitor, Evaluation, Assumption & Dependencies

4.1Technical Evaluation:

4.1.1 Grayscale Rollout & A/B Testing

The team will adopt a Canary Deployment strategy. New agentic logic (e.g., a new version of the Sourcing Analyst's fuzzy matching algorithm) will be released to 5% of procurement officers initially. We will monitor the "Match Accuracy Rate" and "Token Latency." If performance metrics remain stable, the rollout will gradually increase to 100%. A/B Testing will be conducted specifically on prompt templates to determine which "System Role" instructions result in higher quality email drafts with fewer manual edits.

4.1.2 Emergency Rollback (ER) & Golden Release

In the event of a critical failure—such as the GLM-4 API returning malformed JSON that breaks the pipeline—an Automated Emergency Rollback is triggered. The system will revert the backend service to the Golden Release (the last stable version verified to handle basic text parsing). During this state, the system will switch to a "Safe Mode," utilizing local regex extraction instead of LLM extraction to maintain basic functionality.

4.1.3 Priority Matrix:

4.1.3.1

Prioroty	Level	Trigger Condition	Action Taken
P1	Critical	GLM-4 API downtime or >5% failure rate in JSON parsing.	Immediate ER to Golden Release; notify dev team; switch to local fallback logic.
P2	High	Average Token Latency exceeds 15 seconds per request.	Trigger prompt compression logic; scale up API concurrency limits.
P3	Medium	"No Match Found" results exceed 30% of total queries.	Log missed products for manual CSV database

© UMHackathon 2026

			update; trigger RFI (Request for Information) mode.
P3	Low	Minor UI rendering issues or typos in generated drafts.	Log for next sprint; apply hotfix in the next scheduled deployment.

4.2 Monitoring:

4.2.1 We will implement a Health Check Monitor that pings the Zhipu AI API and the local Flask server every 60 seconds.

- Heartbeat Monitoring: If the server fails to respond, an alert is sent via a webhook.
- Semantic Monitoring: A specialized monitor tracks the "Hallucination Rate" by comparing the LLM's identified product names against the suppliers.csv index. If the deviation is too high, the system flags the session for human review.

4.3 Assumptions:

a. a. Procurement officers have access to a stable internet connection to communicate with the external GLM-4 API.
b. The suppliers.csv file is formatted correctly with at least the minimum required headers (Product Name, Price, Email).
c. Users possess basic digital literacy to review and approve the AI-generated email drafts before sending.

4.4 External Dependencies:

Tools	Purpose	Risks
Z Ai GLM API	Core LLM reasoning engine for intent extraction, risk assessment, and email synthesis.	High: API rate limits or service outages. Mitigation: Graceful degradation to local "Safe Mode" extraction.
GLM-4V (Vision)	OCR and visual analysis of physical procurement documents and lists.	Medium: Poor handwriting or image quality may lead to extraction errors. Mitigation: Manual verification UI.
Python-Docx / PyPDF2	Parsing of uploaded .docx and .pdf procurement requirements.	Low: Occasional formatting issues in complex documents.

5. Project Management & Team Contributions

5.1. Project Timeline:

Timeline	Phase	Deliverables & Justification
Day 1-2	Planning & Prompt Engineering	Defined the agentic hand-off logic. Developed the "Procurement Specialist" system prompts for GLM-4 to ensure structured JSON outputs. Justification: Critical to establish a reliable extraction foundation.
Day 3-5	Backend & Data Integration	Developed the Flask API core and the Sourcing Analyst's fuzzy matching logic. Sanitized the suppliers.csv data layer. Justification: Core logic must be stable before building the UI.
Day 6-7	Visual Model & Multi-Modal Support	Integrated GLM-4V for OCR processing and handled file uploads (.docx, .pdf). Justification: Enables the "Multi-Modal" value proposition of the system.
Day 8-9	Frontend & State Visualization	Built the index.html dashboard with real-time "Thinking Process" logs for each agent. Justification: Essential for transparency and "Human-in-the-Loop" review during the demo.
Day 10	Optimization & Bug Fixing	Conducted latency testing on GLM API calls and refined error-handling fallbacks (Graceful Degradation). Justification: Ensures a smooth and professional demo for the judges.

5.2. Team Members Role

Member	Role	Responsibilities
Kam Xin Le	Lead AI Architect	Design of the Multi-Agent Orchestration; Prompt Engineering for GLM-4 and GLM-4V; responsible for the logic hand-off between agents.

© UMHackathon 2026

Tin Li Qi	Full-Stack Developer	Development of the Flask Backend; integration of the "Thinking Process" real-time visualization in index.html; API routing and document upload handling.
Lim Ke Xuan	Technical Writer & Data Lead	Management of the suppliers.csv database; authoring the System Analysis Documentation (SAD); designing the 3NF logical data models.
Tang Yue Li	QA & Presentation Lead	Cross-language validation (EN/MS/ZH); preparing edge-case test scenarios; designing the final presentation slides and project demo scripts.

5.3. Recommendations:

- Documentation-Driven Development: The Documentation Team should define the JSON schemas first (in the SAD) before the Technical Team begins coding. This acts as a "Technical Contract" to prevent integration errors between agents.
- Automated QA Pipeline: Since the Documentation Team is also handling QA, they should provide the Technical Team with a "Golden Dataset" of 10 complex procurement scenarios to verify that updates to the GLM prompts don't break previous successes.
- Shared Knowledge Base: Use a shared repository for suppliers.csv to ensure the Documentation Team is always analyzing the same data that the Technical Team is matching against.

© UMHackathon 2026